

DUSY: Backend & MongoDB Architecture

Notes

High-Yield Viva Revision: Connection Architecture, Data Storage, Code Maps & Live Demo Guide

VIVA REVISION
NOTES

1. What Are the Backend and Database? (Core Concepts)

What is the Backend? (FastAPI)
Built in **Python with FastAPI**. It runs on local port 8000. It acts as the logic engine that manages API routes, verifies user security (JWT tokens), calculates BMI, formats clinical prompts, and coordinates database operations.

What is the Database? (MongoDB)
A **NoSQL document database** running on local port 27017 under the database name health_assistant. It persistently saves all user accounts, clinical baseline profiles, consultations, and lifestyle logs as flexible JSON/BSON documents.

2. Why Did We Use MongoDB Over SQL? (The Key Defense Point)

In technical evaluations, teachers frequently ask: "Why did you choose MongoDB instead of MySQL, Oracle, or PostgreSQL?" Here are the three architectural reasons to state:

- 1. Polymorphic Healthcare Data:** Patient health records vary widely. One patient has 4 allergies and a history of Type-2 diabetes; another has no allergies and high blood pressure. In relational SQL, this requires rigid tables with dozens of empty NULL columns. In MongoDB, documents are flexible and store only what the patient actually possesses.
- 2. Nested Multi-Turn Consultations:** Clinical chat histories contain sequential conversation turns, stage tags, and recovery goals. In MongoDB, these fit naturally into a single document array (messages: [...]) without performing heavy multi-table SQL joins.
- 3. Asynchronous Non-Blocking I/O:** We use the **Motor** driver, which natively pairs with FastAPI's asyncio event loop so that database queries never freeze the web server.

3. Where Is the Data Stored Physically on the Laptop?

Attribute	Value / Exact Path
Database Engine	MongoDB Community Server 8.0
Database Name	health_assistant
Local Port	localhost:27017
Physical Hard Drive Directory	C:\Program Files\MongoDB\Server\8.3\data
Live Stored Collections	7 collections: users, health_profiles, agent_consultations, agent_actions, preventive_analyses, conversations, health_projections

4. Where to See the Database Connections in Code (Exact File Map)

File Path in Project	What It Does (The Connection & Usage Point)
<code>app/config.py</code>	Connection Settings: Loads <code>MONGODB_URI = "mongodb://localhost:27017"</code> and <code>DATABASE_NAME = "health_assistant"</code> from environment variables.
<code>app/database.py</code>	Client Singleton: Imports <code>AsyncIOMotorClient</code> from <code>Motor</code> , creates the database connection pool, and exposes <code>get_database()</code> to all routes.
<code>app/main.py</code>	Server Lifespan Hook: Automatically opens the MongoDB connection pool when FastAPI starts and gracefully closes it when the server stops.
<code>app/services/auth_service.py</code>	User Security & Registration: Hashes passwords with salted Bcrypt and writes user credentials to the <code>users</code> collection.
<code>app/services/health_service.py</code>	Profile Management: Automatically computes BMI and performs upsert operations (<code>update_one</code> with <code>upsert=True</code>) on the <code>health_profiles</code> collection.
<code>app/services/agent_service.py</code>	Context Assembly & Chat Logs: Queries user vitals to inject into Gemini and atomically appends turns into <code>agent_consultations</code> using <code>\$push</code> .

5. How the Connection Lifecycle Operates (Under the Hood)

1. **Initialization:** When uvicorn boots `app/main.py`, the `lifespan` event calls `connect_to_database()` in `app/database.py`.
2. **Socket Connection:** `AsyncIOMotorClient` connects to `mongodb://localhost:27017` and checks connection health using a ping command.
3. **Dependency Injection:** Whenever a request hits an API route, the route imports `get_database()`. The server borrows an active socket from the Motor connection pool, executes the query asynchronously, and returns the socket to the pool.
4. **Graceful Teardown:** When the server terminates, `close_database_connection()` shuts down open sockets, preventing connection leaks.

6. How to Demonstrate Stored Data Live to Evaluators (3 Methods)

Method 1: MongoDB Compass (Visual UI)

1. Open **MongoDB Compass** from Windows Start.
2. Click green **Connect** (to `localhost:27017`).
3. Click database **health_assistant**.
4. Click on **health_profiles** or **agent_consultations** to show live expandable JSON documents.

Method 2: One-Click Terminal Viewer

1. Open terminal in the project folder.
2. Run: `python show_database.py`
3. It prints the disk path, all 7 collection counts, and live records: **Height 185 cm, Weight 70 kg, BMI 20.5, Blood B+, Peanut Allergy**, and 13 consultations.

7. What to Say in Your Viva (Word-for-Word Speech)

*"Respected teachers, our backend is built on **FastAPI in Python**, and all clinical data is persistently stored in a local **MongoDB Community Server** instance under the database `health_assistant`.*

We specifically selected MongoDB over SQL because healthcare telemetry is polymorphic and semi-structured — patients possess varying sets of allergies, preconditions, lifestyle metrics, and multi-turn consultation transcripts. In MongoDB, these fit naturally into native JSON documents without dozens of empty NULL columns or expensive table joins.

*To connect FastAPI with MongoDB, we used the **Motor asynchronous driver** (`AsyncIOMotorClient`). Motor runs on FastAPI's `asyncio` event loop, ensuring that database read and write operations never block server execution threads.*

*All user passwords are encrypted using salted **Bcrypt hashing**, and all database files are stored on disk at `C:\Program Files\MongoDB\Server\8.3\data`. We can inspect any stored document live in MongoDB Compass or through our Python inspection utility."*

8. Quick-Fire Teacher Viva Q&A (Memorize These 4 Points)

Q1: Why did you use Motor instead of standard PyMongo?

Answer: "Standard PyMongo is synchronous and blocks the Python thread during I/O operations, which slows down FastAPI. Motor is asynchronous and non-blocking, allowing the server to handle concurrent user requests smoothly."

Q2: Where is user authentication verified before database access?

Answer: "In `app/middleware/auth_middleware.py`, we use a `get_current_user` dependency that extracts and validates the JWT Bearer token from the request header before any protected database query is executed."

Q3: How do you prevent chat messages from overwriting existing history?

Answer: "We use MongoDB's atomic `$push` operator in `agent_service.py`. This appends new user and AI messages to the existing messages array within the consultation document without rewriting the entire record."

Q4: How do you ensure data integrity without SQL schemas?

Answer: "We use **Pydantic v2** data models in FastAPI. Pydantic validates data types, calculates BMI automatically, and enforces required fields before data is ever sent to MongoDB."